

Expense Tracker Application

AI-assisted product engineering report covering delivery methodology, architecture, implementation quality, deployment readiness, and forward-looking modernization opportunities for senior technology leadership.

This document positions the Expense Tracker as a working digital finance product, built with a modern React frontend, deployed through Vercel CI/CD, and accelerated through Cursor AI under human engineering review. It highlights both delivered capability and the technical path toward a scalable cloud-backed platform.

DEVELOPMENT MODEL

Cursor AI-assisted development with human review, validation, and modification.

APPLICATION TYPE

React-based personal expense tracking web application.

PRODUCTION DEPLOYMENT

[Vercel live application](#)

SOURCE REPOSITORY

[GitHub repository](#)

Table of Contents

1. Executive Summary	3	9. Feature Implementation Details	15
2. Project Objectives	4	10. Cursor AI Usage Analysis	16
3. Functional Requirements	5	11. Challenges and Resolutions	17
4. Non-Functional Requirements	7	12. Testing Strategy	18
5. Technology Stack	8	13. Deployment Process	19
6. System Architecture	10	14. Metrics and Outcomes	20
7. AI-Assisted Development Methodology	13	15. Future Enhancements	21
8. Development Workflow	14	16. Conclusion	22

1. Executive Summary

1.1 Project Overview

The Expense Tracker Application is a modern, responsive web application designed to help users record, manage, analyze, and export personal expense data. The application supports expense entry, category management, dashboard analytics, history filtering, Excel export, budget-related features, recurring expenses, receipt scanning, voice entry, and multi-currency expense normalization.

The application was developed primarily using Cursor AI, with human review, validation, and modifications applied throughout the development lifecycle. Cursor AI was used as an engineering accelerator for code exploration, implementation, refactoring, debugging, validation support, and documentation.

1.2 Business Objective

The primary business objective is to provide users with a simple and intelligent way to track spending behavior, understand financial patterns, and make informed decisions through organized history, charts, reporting, and export functionality.

The project also demonstrates how AI-assisted development can accelerate delivery while still requiring structured human oversight, validation, and technical judgment.

1.3 Key Outcomes

React
Component-driven frontend

Excel
Exportable reports

AI
Assisted development

FX
Multi-currency support

- A functional React-based expense tracking application was implemented.
- Users can add, edit, delete, filter, and export expenses.
- Dashboard analytics provide category-wise spending insights.
- Category and budget management improve personalization.
- Multi-currency support converts entered expenses into the configured setup currency.
- OCR and voice-assisted expense entry capabilities were introduced.
- The application follows a component-driven frontend architecture.

2. Project Objectives

2.1 Problem Statement

Users frequently track expenses manually across spreadsheets, notes, or disconnected tools. These approaches can become difficult to maintain, lack meaningful analytics, and do not provide a unified view of spending behavior. The application addresses this problem by providing a centralized, lightweight, and user-friendly expense management tool.

2.2 Target Users

- Individuals tracking personal expenses.
- Students managing monthly spending.
- Professionals monitoring daily and recurring costs.
- Users requiring category-based financial visibility.
- Users who need lightweight reporting without a complex accounting platform.

2.3 Scope of the Application

In Scope	Out of Current Scope
Expense creation, edit, deletion, and restore	Production backend API
Category and budget setup	Centralized cloud database
Dashboard analytics and charts	Payment integration
Search, filtering, and Excel export	Enterprise reporting platform
Voice, OCR, and multi-currency entry	Native mobile app

3. Functional Requirements

3.1 Expense Management

The application supports the following expense operations:

- Add a new expense with name, amount, category, date, status, and currency.
- Edit an existing expense and re-normalize currency values when needed.
- Delete an expense with confirmation.
- Restore recently deleted expenses through undo behavior.
- Mark expenses as paid or unpaid.
- Convert entered currency into the user’s configured setup currency.

3.2 Category Management

Users can manage expense categories to align the application with their personal spending model. Categories are reused across forms, filters, charts, exports, and budget management.

3.3 Search and Filtering

Filter	Purpose
Text search	Find expenses by name, category, date, or amount.
Category	Isolate spending within one category.
Status	View paid, unpaid, or all expenses.
Date range	Analyze expenses for a selected period.
Amount range	Find low-value or high-value transactions.

3.4 Dashboard and Analytics

The dashboard provides visual insights into expense data through category-wise totals, overall expense amount, budget summaries, and chart visualizations. The pie chart includes the overall expense amount in the center to improve executive readability and immediate context.

3.5 Reports and Export

Reporting is provided through filtered history views and Excel export. Exported files include normalized amounts as well as original currency information, allowing users to audit exchange-rate based conversions.

3.6 User Experience Requirements

- Responsive layout for desktop and mobile usage.
- Clear sidebar and mobile bottom navigation.
- Card-based design system.
- Animated UI transitions where appropriate.
- Accessible form controls and labels.
- Clear empty states and inline validation messages.
- Consistent currency and date formatting.

4. Non-Functional Requirements

Requirement	Implementation Consideration
Performance	Frontend-only local reads and writes, memoized chart calculations, Vite production build, and focused component rendering.
Scalability	Suitable for single-user browser-local usage; future cloud scale requires backend APIs, database, and synchronization.
Maintainability	Component-driven architecture, reusable hooks, and utility modules for storage, currency, analytics, OCR, and parsing.
Security	No payment data is processed; local storage is not encrypted; cloud sync would require secure authentication and encryption.
Responsiveness	Mobile-friendly navigation, responsive forms, flexible chart containers, and adaptive list layouts.

Assumption: The current implementation is a client-side application. It does not currently include a production backend, centralized database, or formal enterprise authentication service.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React	Component-based UI development.
Build Tool	Vite	Fast local development and production builds.
Routing	React Router	Page routing and protected route flows.
Charts	Recharts	Dashboard and expense analytics visualization.
OCR	Tesseract.js	Receipt and bill image text extraction.
Excel Export	ExcelJS	Export expense reports to `.xlsx` files.
Storage	Browser localStorage	Client-side persistence.
State Management	React Hooks	Local and shared UI state.
Version Control	Git and GitHub	Source control and remote repository management through GitHub .
Deployment	Vercel	Continuous deployment of the React/Vite application to Vercel production hosting .
AI Tooling	Cursor AI	AI-assisted coding, review, refactoring, debugging, and documentation.

Backend Technologies

No production backend is currently implemented. The application is primarily frontend-driven and persists data in browser storage utilities.

Database

No external database is currently used. Data is persisted through local browser storage. For production multi-device usage, a cloud database should be introduced.

Deployment Platform

The application is deployed on Vercel using a CI/CD workflow connected to the GitHub repository. Changes pushed to the configured branch can be built and deployed automatically, enabling rapid preview validation and production release through a managed static hosting platform.

Technical Implementation Notes

Area	Technical Detail
Authentication	Client-side sign up, sign in, sign out, protected routes, and session persistence. Passwords are stored as SHA-256 hashes in browser storage for demo purposes.
Data Isolation	Storage records are keyed by user ID for expenses, categories, budgets, preferences, and recurring rules.
Currency Normalization	Expense amounts are normalized to the configured setup currency while preserving original amount, currency, exchange rate, and rate date.
Reporting	Excel export includes normalized values, source currency fields, and total rows for auditability.
Deployment Pipeline	Vercel CI/CD performs install, build, static asset generation, and deployment from GitHub source control.

6. System Architecture

6.1 High-Level Architecture Diagram

Mermaid source:

```
graph TD
    User[User] --> Browser[Browser / React Application]
    Browser --> Router[React Router]
    Router --> Layout[Main Layout]
    Layout --> Dashboard[Dashboard Page]
    Layout --> AddExpense[Add Expense Page]
    Layout --> History[History Page]
    Layout --> Settings[Settings Page]
    AddExpense --> ExpenseForm[Expense Form]
    AddExpense --> VoiceDialog[Voice / Bill Confirmation Dialog]
    AddExpense --> OCR[Tesseract OCR]
    AddExpense --> NLP[NLP Expense Parser]
    ExpenseForm --> ExchangeRate[Exchange Rate Service]
    VoiceDialog --> ExchangeRate
    Dashboard --> Analytics[Expense Analytics Utilities]
    History --> Export[Excel Export Utility]
    Settings --> Preferences[User Preferences]
    Analytics --> Storage[Browser Local Storage]
    Preferences --> Storage
    Export --> Storage
    ExchangeRate --> PublicAPI[Public Currency Rate API]
```

6.2 Component Interaction Diagram

flowchart LR

```
MainLayout[MainLayout] --> TopBar[TopBar]
MainLayout --> SideNav[SideNav]
MainLayout --> MobileBottomNav[MobileBottomNav]
MainLayout --> Dashboard[Dashboard]
MainLayout --> AddExpense[AddExpense]
MainLayout --> History[History]
MainLayout --> Settings[Settings]
AddExpense --> ExpenseForm[ExpenseForm]
AddExpense --> ExpenseChart[ExpenseChart]
AddExpense --> RecurringManager[RecurringManager]
ExpenseForm --> VoiceExpenseDialog[VoiceExpenseDialog]
ExpenseForm --> CurrencyUtils[Currency Utilities]
ExpenseForm --> ExchangeUtils[Exchange Rate Utilities]
History --> ExpenseList[ExpenseList]
ExpenseList --> ExpenseItem[ExpenseItem]
ExpenseItem --> ExpenseEditDialog[ExpenseEditDialog]
ExpenseItem --> ConfirmationDialog[ConfirmationDialog]
Settings --> CategoryManager[CategoryManager]
Settings --> CategoryBudgetManager[CategoryBudgetManager]
```

6.3 Data Flow Diagram

flowchart TD

```
A[User enters expense] --> B[Validate form data]
B --> C{Currency same as setup currency?}
C -->|Yes| D[Store amount as normalized amount]
C -->|No| E[Fetch historical exchange rate]
E --> F[Convert amount to setup currency]
F --> G[Store normalized amount and original currency metadata]
D --> G
G --> H[Persist to localStorage]
H --> I[Dashboard totals update]
H --> J[History list updates]
H --> K[Charts update]
H --> L[Export data available]
```

6.4 Application Layers

Layer	Responsibility
Presentation Layer	React components, pages, dialogs, navigation, forms, and charts.
State Layer	React hooks for expenses, categories, budgets, preferences, and recurring expenses.
Utility Layer	Currency formatting, exchange rates, storage, analytics, OCR, and NLP parsing.
Persistence Layer	Browser localStorage.
Integration Layer	Public exchange-rate API, OCR processing, and Excel export.

7. AI-Assisted Development Methodology

7.1 Requirement Gathering

Requirements were captured through iterative user prompts. Cursor AI interpreted requested features, inspected the codebase, identified affected modules, and proposed implementation changes. Human oversight ensured that each implementation aligned with intended product behavior.

7.2 Prompt Engineering Strategy

Prompt Type	Example	Outcome
Feature Request	Add multi currency feature while adding an expense.	Currency fields, exchange-rate normalization, and metadata storage.
UI Enhancement	Add overall expense in the middle of the pie chart.	Dashboard chart total overlay.
Debugging	It is not showing yet.	Replacement of fragile Recharts label with HTML overlay.
Documentation	Generate a technical implementation report.	Senior-management-facing project report.

7.3 Code Generation Using Cursor

- Read relevant files and understand component dependencies.
- Modify React components and CSS files.
- Add utility modules such as exchange-rate normalization.
- Wire feature logic through layout and hooks.
- Run diagnostics and production build validation.

7.4 Code Review Process

Review included reading affected files before edits, keeping changes scoped, checking state flow, validating import paths, running linter diagnostics, and verifying production builds where appropriate.

7.5 Human Oversight

Human oversight remained essential for confirming expected behavior, reviewing generated code, validating business intent, identifying UI gaps, and requesting corrections.

8. Development Workflow

flowchart LR

Planning[Planning] --> Design[Design]

Design --> Development[Development]

Development --> Testing[Testing]

Testing --> Review[Human Review]

Review --> Deployment[Deployment]

9. Feature Implementation Details

9.1 Expense Management

Area	Description
Objective	Allow users to create and manage expenses.
Design Approach	Reusable form and list components with local state and storage hooks.
Implementation	Expense data is created from forms and persisted through user expense hooks.
Challenge	Maintaining consistent behavior across add, edit, voice, and bill flows.
Resolution	Route expense creation and update through normalized handlers in layout.

9.2 Multi-Currency Expense Entry

Area	Description
Objective	Allow users to enter expense currency and convert to setup currency.
Design Approach	Keep `amount` as normalized setup-currency amount and store original metadata.
Implementation	Added exchange-rate utility and currency fields in add/edit dialogs.
Challenge	Preserving existing dashboard and history calculations.
Resolution	Existing analytics continue using normalized `expense.amount`.

Stored currency-related fields include amount, currencyCode, originalAmount, originalCurrencyCode, exchangeRate, and exchangeRateDate.

9.3 Category Management

Category management supports custom category structures, category-aware forms, filters, charts, and budget allocation. The implementation keeps category logic separate from expense storage while allowing dashboards to include visible categories derived from configured categories and existing data.

9.4 Dashboard and Analytics

Dashboard analytics use category totals to populate Recharts visualizations. The center of the donut chart displays the overall expense value through a styled HTML overlay, improving reliability and readability.

9.5 Export Functionality

Excel export uses ExcelJS to create a workbook with expense rows, total amount, and currency audit fields. This supports offline review, archival, and stakeholder reporting.

10. Cursor AI Usage Analysis

10.1 Usage Summary

Cursor AI was used throughout the project for codebase exploration, implementation, refactoring, debugging, validation support, and documentation. It served as an engineering accelerator rather than a replacement for engineering judgment.

Area	AI-Assisted Impact
Codebase Understanding	Faster file discovery and dependency tracing.
Implementation	Reduced time to wire feature flows across components.
Debugging	Faster identification of prop-flow and rendering issues.
Refactoring	Improved reuse of existing utilities and patterns.
Documentation	Significant reduction in time required to produce formal reports.

10.2 Advantages

- Rapid iteration and reduced boilerplate coding.
- Improved discovery of impacted files.
- Helpful validation through build and lint execution.
- Faster documentation and explanation generation.

10.3 Limitations

- AI may initially choose a technically valid but visually ineffective implementation.
- AI requires human validation for UX expectations and business intent.
- External API reliability must still be evaluated by engineers.
- Security and scalability require architectural review beyond code generation.

10.4 Risk Mitigation

Risk was mitigated through human review, lint and build checks, scoped edits, reuse of existing project patterns, and explicit documentation of assumptions where implementation details were unavailable.

11. Challenges and Resolutions

Challenge	Resolution
Multi-currency conversion without disrupting analytics	Kept normalized setup-currency value in `amount`.
Historical exchange-rate lookup	Added exchange-rate utility based on selected expense date.
Chart center label not visible	Replaced Recharts label with reliable HTML overlay.
Multiple entry paths	Routed manual, edit, voice, and bill flows through common normalization.
Export auditability	Added original currency and exchange-rate fields to Excel export.

12. Testing Strategy

12.1 Current Validation

Validation included IDE diagnostics, linter checks, production build validation, manual UI review, and targeted correction of discovered behavior gaps.

12.2 Recommended Automated Testing

Test Area	Examples
Unit Testing	Currency formatting, exchange-rate normalization, storage utilities, filtering, category totals, parsing utilities.
Functional Testing	Add, edit, delete, restore, filter, export, and currency conversion flows.
Manual Testing	Responsive layout, chart rendering, Excel export verification, voice and bill confirmation review.
Edge Cases	Missing fields, zero amounts, future dates, unsupported currency rates, empty expense list, no matching filters.
Performance Testing	Large local expense lists, chart rendering, history filter responsiveness, Excel export volume.

13. Deployment Process

13.1 Build Process

The application uses Vite. The production build command is:

```
npm run build
```

13.2 CI/CD Approach

The project is connected to GitHub source control and deployed through Vercel CI/CD. The live application is available at <https://reactexpensetracker-eight.vercel.app/>, and the source repository is available at https://github.com/Vignesh-Git/React_Expense_Tracker.git.



13.3 Production Deployment Summary

Area	Current Implementation
Hosting	Vercel static frontend hosting.
Source Control	GitHub repository integration.
Build Command	npm run build.
Output	Vite-generated static assets from the dist directory.
Release Model	Continuous deployment from GitHub to Vercel with preview and production environments.

14. Metrics and Outcomes

Assumption: Exact time tracking was unavailable. The observations below represent qualitative assessment based on the development workflow.

Activity	AI-Assisted Impact
Code exploration	High time savings.
UI implementation	High time savings.
Refactoring	Medium to high time savings.
Debugging	Medium time savings.
Documentation	High time savings.
Testing support	Medium time savings.

14.1 Code Quality Observations

- Component-based structure supports maintainability.
- Utility functions reduce duplication.
- Existing formatting utilities were reused.
- Feature changes were scoped to relevant files.
- Build validation was used after meaningful changes.

14.2 Improvement Opportunities

- Add automated unit and integration tests.
- Add CI/CD with mandatory lint and build checks.
- Add backend and secure persistence.
- Add exchange-rate caching.
- Add runtime error monitoring.

15. Future Enhancements

Enhancement	Description
AI-powered receipt scanning	Extract merchant, tax, total, date, currency, line items, and category suggestions.
Budget forecasting	Predict monthly spend and budget burn rate from historical patterns.
Smart categorization	Recommend categories based on merchant, description, and prior behavior.
Financial insights	Month-over-month comparison, unusual expense detection, and saving opportunities.
Mobile support	Progressive Web App or React Native mobile application.
Cloud synchronization	Secure multi-device sync, backup, restore, and API-driven reporting.

16. Conclusion

The Expense Tracker Application demonstrates a practical and effective use of AI-assisted software development. Cursor AI accelerated implementation, refactoring, debugging, and documentation while human review ensured that business intent, UX expectations, and technical quality were preserved.

The application provides meaningful expense management capabilities, including category tracking, dashboard analytics, export functionality, voice and receipt assisted entry, and multi-currency conversion. The current architecture is suitable for a frontend-first personal finance application and can be extended into a scalable cloud-backed solution.

Recommendations for Future AI-Assisted Development Projects

- Use AI for rapid implementation while retaining human architectural ownership.
- Require lint, build, and test validation for AI-generated changes.
- Maintain clear coding standards and reusable design patterns.
- Use AI for documentation and test-case generation.
- Track productivity metrics to quantify AI impact.
- Review AI-generated code for security, maintainability, and long-term scalability.

Prepared as a management-ready report based on the current React Expense Tracker implementation and documented assumptions where project information was unavailable.